CAS4160 Homework 7: RLHF

[Your name]
[Your student ID]

1 Introduction

The goal of this assignment is to understand and gain hands-on experience with Reinforcement Learning from Human Feedback (RLHF). Unlike traditional reinforcement learning algorithms that rely on predefined environment rewards, RLHF learns a reward model from human feedback, and then uses that learned reward to train a policy.

While RLHF is widely known for its use in training large language models (LLMs), the original research was actually conducted in the context of robot control.

In this assignment, you will implement an RLHF framework by extending the PPO algorithm we previously developed. Instead of using an environment-provided reward, you will define a reward function based on human preferences, and train this reward model accordingly. You will then provide feedback directly to guide the robot's behavior, giving you a practical understanding of how RLHF works and having fun to see robot evolving.

2 Installation

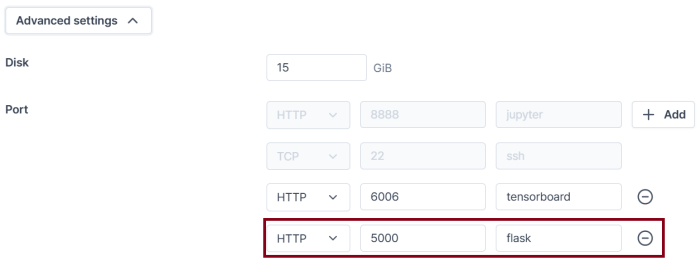

Figure 1: Open port 5000 for Flask

In this assignment, unlike previous ones, you will need to install additional packages. Please make sure to check the `requirements.txt` file and install all required dependencies before starting.

Additionally, to provide feedback (preference), we will use a web framework called Flask. If you are using VESSL AI, make sure to open additional ports to allow the web interface to function properly. Please refer to Figure 1 for more detail. Please use the Chrome browser for this.

3 Code structure overview

The code structure is similar to that of the HW2, but we are going to add few more parts for RLHF. In this assignment, you mainly work with the following codes:

- `cas4160/networks/reward_predictor.py`: Reward model implementation.
- `cas4160/agents/pg_agent.py`: PPO agent with the learned reward model.
- `cas4160/scripts/run_hw7.py`: The main training loop for your implementation.

4 Reinforcement learning from human feedback

We will provide a brief overview of Reinforcement Learning from Human Feedback (RLHF), followed by implementation. You will: (1) define a reward function and implement the reward learning objective, (2) collect human-annotated preference samples, store them in a replay buffer, and train the reward model, and finally, (3) use the learned reward to train a PPO agent.

4.1 Formulation of preference

We consider an agent interacting with an environment over a sequence of steps. At each timestep t, the agent observes a state $\mathbf{s}_t \in \mathcal{S}$ and selects an action $\mathbf{a}_t \in \mathcal{A}$. In standard RL, the environment returns a scalar reward $r_t \in \mathbb{R}$, and the agent's goal is to maximize the expected cumulative discounted reward.

In contrast, we assume that the environment does not provide explicit rewards. Instead, a human overseer gives *preference comparisons* between short trajectory segments. A trajectory segment is defined as a sequence:

$$\tau = ((\mathbf{s}_0, \mathbf{a}_0), (\mathbf{s}_1, \mathbf{a}_1), \ldots, (\mathbf{s}_{k-1}, \mathbf{a}_{k-1})).$$

We denote $\tau_1 \succ \tau_2$ to indicate that the human prefers segment τ_1 over τ_2.

Goal. The agent should generate trajectories preferred by the human, while minimizing the number of preference queries.

Evaluation criteria.

- **Quantitative evaluation:** Assume preferences are induced by an unknown reward function $r : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$. Then $\tau_1 \succ \tau_2$ implies:
$$\sum_{t=0}^{k-1} r(\mathbf{s}_t^1, \mathbf{a}_t^1) > \sum_{t=0}^{k-1} r(\mathbf{s}_t^2, \mathbf{a}_t^2).$$
If r is known, we can evaluate agents based on their true cumulative rewards.
- **Qualitative evaluation:** When r is unknown, we evaluate agents based on how well their behavior aligns with human preferences. This involves presenting a natural language goal to a human and asking for evaluations based on observed agent behavior videos.

In our setup, the human overseer is presented with a visualization of two trajectory segments in the form of short movie clips, each lasting between 1 to 2 seconds.

The human annotator provides feedback by indicating one of the following:

- τ_1 is preferred than τ_2.
- τ_2 is preferred than τ_1.
- Both segments are equally good.

Each comparison is stored as a triplet (τ_1, τ_2, μ) in a replay buffer $\mathcal{D}$, where:

- τ_1 and τ_2 are the two compared trajectory segments,
- μ is a probability distribution over $\{1, 2\}$ representing the user's preference.
- If the user prefers one segment, μ assigns all mass to that segment.
- If both segments are marked as equally preferable, then $\mu = (0.5, 0.5)$.

4.2 Fitting reward function

The estimated reward function $\hat{r}$ can be considered as a latent variable explaining human judgments, a **preference function** $\hat{P}$. We model the probability that a human prefers a trajectory segment τ_1 over τ_2 as:

$$\hat{P}[\tau_1 \succ \tau_2] = \frac{\exp\left(\sum \hat{r}(\mathbf{s}_t^1, \mathbf{a}_t^1)\right)}{\exp\left(\sum \hat{r}(\mathbf{s}_t^1, \mathbf{a}_t^1)\right) + \exp\left(\sum \hat{r}(\mathbf{s}_t^2, \mathbf{a}_t^2)\right)}, \tag{1}$$

where the summation is taken over all timesteps within each segment.

To train $\hat{r}$, we minimize the cross-entropy loss between this predicted preference and the actual human-labeled preference distribution μ:

$$\mathcal{L}(\hat{r}) = -\sum_{(\tau_1, \tau_2, \mu) \in \mathcal{D}} \Big[\mu(1) \log \hat{P}[\tau_1 \succ \tau_2] + \mu(2) \log \hat{P}[\tau_2 \succ \tau_1]\Big] \tag{2}$$

4.3 Implementation

Let's implement RLHF step by step:

1. `cas4160/networks/reward_predictor.py`: Implement all TODOs in this file to train a reward model.
2. `cas4160/agents/pg_agent.py`: This code implements PPO, but you need to make it to use the reward function implemented above by changing only one line of code.
3. `cas4160/scripts/run_hw7.py`: You need to implement to save human annotated triplets to a replay buffer and to use it for reward model training, and to provide reward signals to the PPO agent. Implement all TODOs in this file.

5 Experiments

5.1 PointMaze with synthetic preferences

- This task is designed to help debug your code and understand RLHF and feedback mechanisms. In the maze, there is a green ball and a red target. The goal is for the green ball to move toward the red target. You should provide feedback by "preferring" the trajectory that appears more likely to reach the target based on the green ball's motion.
- Let's first train the reward model using synthetic preferences. Instead of receiving human feedback, the environment's reward is used to generate synthetic preferences, which are then used for training. Take a look at how this works in `run_hw7.py` when `syn_prefs` is set to **True**. Run the bash file below to train a model at `PointMaze_UMazeDense-v3` on synthetic preferences.

```
sh pointmaze_syn_prefs.sh
```

- After training is complete, open TensorBoard to check the training results. Note that you can also download rollout gifs from tensorboard.
- **Question:** Check the training results and evaluation rollouts. How did the trained agent perform? Explain why the results are not optimal, based on the reward, and the structure of the maze. And attach the rollout frames at Figure 2.

  Answer:

  Add your explanation.

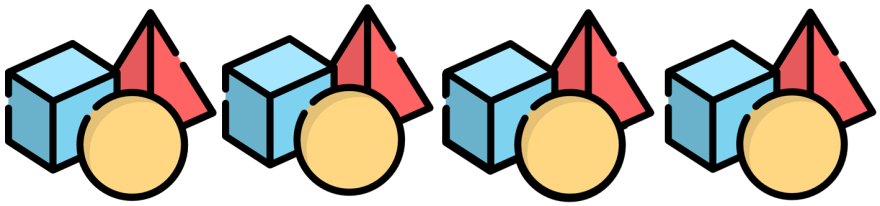

Figure 2: Point maze rollout with synthetic preferences. **Replace this with your plot, and add the caption.**

5.2 PointMaze with human preferences

- This time, let's solve the maze more effectively using your feedback. This bash script runs the code that includes the trajectory annotation GUI.

```
sh pointmaze.sh
```

- If you launch the bash file, you can see these outputs from terminal:

```
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://***.***.***.***:5000
 Press CTRL+C to quit
 Using GPU id 0

 ********** Iteration 0 ************

 Collecting video rollouts for feedback...
```

  This means your training run has started. Now, you can provide preference feedback via your web browser at `127.0.0.1:5000`. Note that if you are using VESSL AI, you should navigate to metadata panel and click port `5000` as you open the port for TensorBoard. Then, the web browser will show you the GUI interface, as shown in Figure 3. You can click buttons or use arrow keys in your keyboard to provide feedback.

Figure 3: Web GUI for human preference feedback.

- Once training is complete, you can use TensorBoard to visualize the results. Since this task is very simple, training should finish in about 10 minutes. If you provide reasonable feedback, you will see the green ball moving smoothly toward the red goal without any errors.

Here are some tips for giving feedback:

- The videos you see are not the whole trajectory of policy rollout. To speed up training, we only show the last few seconds.
- Make some concrete criteria internally and give consistent feedback during an experiment.
- Nothing is perfect on the first try. Choose the trajectory that shows even a slight movement toward the goal. It will gradually improve over time.
- Avoid choosing 'Neutral' unless absolutely necessary. Any small differences can lead to big changes. Select 'Neutral' only when it is truly hard to distinguish between the two trajectories.
- If your trained model shows no potential to follow the intended goal, restart the training.
- If the same action was taken, the one performed faster is better.

Question. Please provide the successful rollout frames of your trained agent in Figure 4. Explain why human feedback is beneficial for this task, and discuss the conditions under which RLHF becomes necessary or particularly helpful for policy learning.

Answer:

Add your explanation.

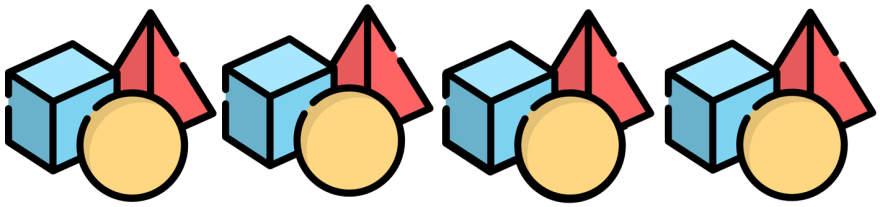

Figure 4: Point maze rollout with human preferences. **Replace this with your plot, and add the caption.**

5.3 Hopper Backflip

Alright. But, it is not too surprising. Shall we try something a bit more interesting? In this section, we will use the Hopper environment, where we have tested various algorithms before, to attempt a backflip. Imagine trying to design a reward function that makes Hopper perform a backflip–sounds tough, right? But with RLHF, just a bit of your quality feedback and a little more time is all it takes to get Hopper to do a backflip!

- Train a model on `Hopper-v5`. This bash script changes to the execution directory and runs the code that includes the trajectory annotation GUI.

  ```
  sh hopper.sh
  ```

- This experiment performs 100 optimization steps while asking for human feedback every 5 steps. The entire experiment will approximately take 1 hour and 20 minutes in VESSL AI.

- We include *hopper_flip.mp4* in the homework folder to show the example of hopper learning to do a backflip. You can give a feedback based on that video. Show your prompting skills!

- A backflip is considered successful if the hopper completes one backward revolution. Contact with the grounding during the motion is allowed.

Question. Please provide the rollout of your trained policy in Figure 5 by sampling a few frames from the saved video. Plot `Eval_AverageReturn` of your policy to Figure 6. Finally, include the last successful rollout video as `[Student_ID]_Hopper_Backflip.gif` inside the submission zip file.

Figure 5: Hopper backflip rollout. **Replace this with your plot, and add the caption.**

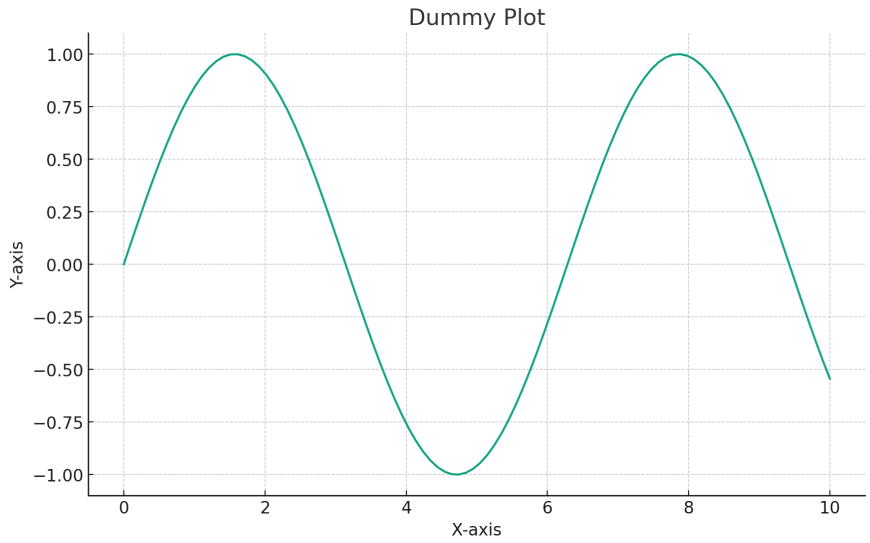

Figure 6: Hopper backflip average return. **Replace this with your plot, and add the caption.**

6 Submission

Please submit the code, a backflip gif, and the **"report"** in a single zip file, `hw7_[YourStudentID].zip`. The zip file should be smaller than 50MB. The structure of the submission file should be:

```
hw7_[YourStudentID].zip
├─ hw7_[YourStudentID].pdf
├─ [YourStudentID]_Hopper_Backfilp.gif
├─ cas4160
│  ├─ agents
│  │  └─ pg_agent.py
│  ├─ networks
│  │  ├─ reward_predictor.py
│  │  └─ ...
│  └─ ...
└─ ...
```

We hope you enjoy this assignment, and thank you for all your hard work throughout the semester. We also hope these homework assignments have deepened your understanding of reinforcement learning!